

TECHNICKÉ ZADÁNÍ PRO AI: Refaktorce BTC Pay Lite – škálovatelný multi-wallet Electrum + XPUB + oddělený payment monitoring

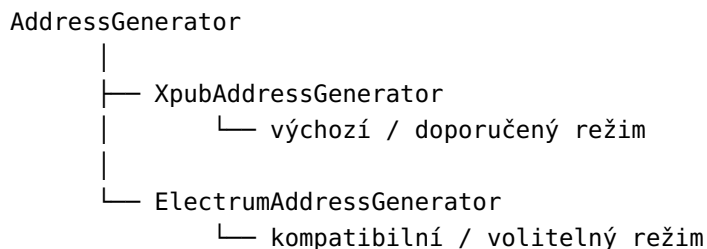
Pracuji na projektu **BTC Pay Lite**, což je lehký PHP Bitcoin payment server / backend pro směnárnu. Projekt aktuálně používá Electrum daemon přes JSON-RPC, podporuje databázové faktury i stateless faktury a obsahuje Greenfield-like API, webhooks, admin rozhraní a správu více walletů.

Chci provést **architektonickou refaktoraci celého systému**, ale nechci projekt zahodit ani přepisovat od nuly. Cílem je zachovat existující funkčnost a API kompatibilitu, odstranit současná úzká hrdla, bezpečně podporovat mnoho současných požadavků a přidat XPUB jako výchozí způsob generování nových platebních adres.

Při práci nejprve prostuduj všechny dodané soubory projektu a pochop jejich vzájemné vazby. Poté implementuj změny systematicky a konzervativně.

1. HLAVNÍ CÍL ARCHITEKTURY

Výsledný systém má podporovat dva nezávislé mechanismy generování Bitcoin adres:



Výchozí režim má být:

XPUB / output descriptor
→ lokální derivace adresy
→ žádné Electrum RPC při vytvoření faktury

Volitelný režim:

Electrum wallet
→ explicitní požadavek
→ createnewaddress

Současné musí být odděleno:

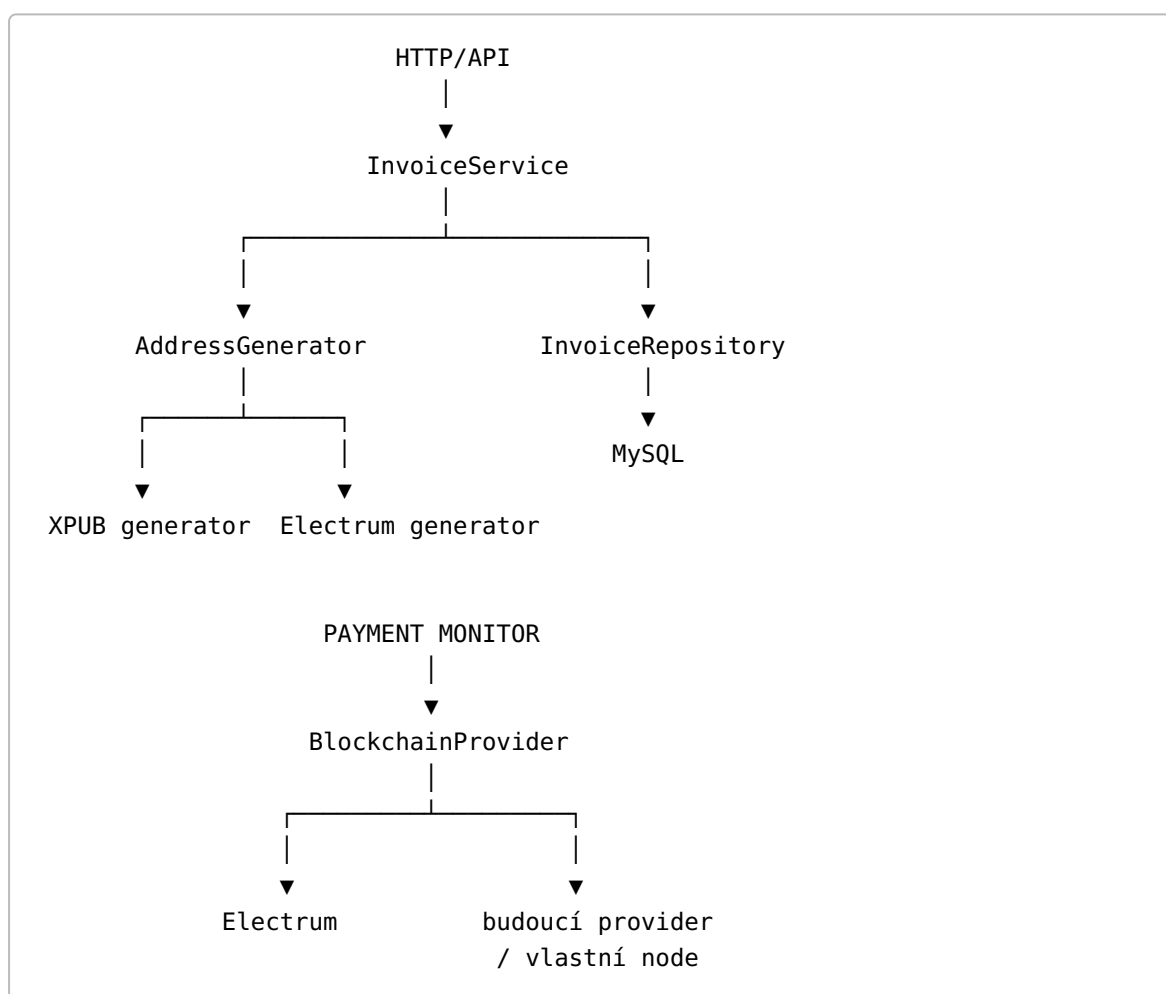
GENEROVÁNÍ ADRES

od:

SLEDOVÁNÍ BLOCKCHAINU / PAYMENT STATUS

`BtcInvoiceManager` nesmí být pevně svázán s `ElectrumWallet` pro obě tyto činnosti.

Cílová architektura přibližně:



Nejdůležitější architektonické pravidlo:

Běžný HTTP request nemá automaticky znamenat RPC dotaz do Electra.

Systém má být navržen tak, aby většina HTTP provozu končila v:

PHP
↓
DB / cache

a pouze malá kontrolovaná část provozu pokračovala do:

Electrum RPC

2. SOUČASNÉ PROBLÉMY, KTERÉ JE POTŘEBA ODSTRANIT

2.1 ElectrumRPC drží globální stav aktivní wallet

Současná třída `ElectrumRPC.php` obsahuje:

```
private string $activeWallet = '';

public function setWallet(string $walletPath): void
{
    $this->activeWallet = $walletPath;
}
```

a následně mění URL:

```
if ($this->activeWallet !== '') {
    $url = rtrim($url, '/') . '/?wallet=' . rawurlencode($this->activeWallet);
}
```

Tento mechanismus odstranit.

RPC klient nesmí mít mutable globální stav typu:

`activeWallet`

protože je to špatná abstrakce pro paralelní/multi-wallet provoz.

Každý wallet-scoped RPC request musí explicitně určit wallet, ke které patří.

Navrhni například:

```
final class ElectrumRPC
{
    public function callDaemon(
        string $method,
        array $params = []
    ): mixed;

    public function callNetwork(
        string $method,
        array $params = []
    ): mixed;

    public function callWallet(
        string $walletPath,
        string $method,
        array $params = []
    ): mixed;
}
```

Implementace nesmí používat:

```
setWallet()
```

ani:

```
activeWallet
```

3. OCHRANA PROTI ROZDÍLŮM MEZI VERZEMI ELECTRA

Neimplementuj wallet routing naslepo.

Nejdříve zjisti přesnou verzi Electrum daemonu, pro kterou projekt běží, například pomocí:

```
version
version_info
getinfo
commands
```

a ověř odpovídající upstream implementaci.

Electrum 4.5.8 používá pro wallet-scoped commandy argument `wallet`, který může obsahovat cestu k již načtené wallet.

Novější Electrum používá interně mechanismus založený na `wallet_path`.

Proto vytvoř kompatibilní vrstvu, například:

```
interface ElectrumRpcDialect
{
    public function walletParams(
        string $walletPath,
        array $params
    ): array;
}
```

nebo jednodušší interní mechanismus v `ElectrumRPC`.

Například:

```
Electrum 4.5.x:
    wallet = /path/to/wallet

novější Electrum:
    wallet_path = /path/to/wallet
```

Přesný parametr musí vycházet z verze, kterou používá náš server.

Nedělej předpoklad, že query string:

```
/?wallet=...
```

je správným mechanismem.

Po implementaci musí existovat integrační test:

```
load wallet A
load wallet B

call wallet A → getbalance
call wallet B → getbalance

ověřit, že každý request skutečně pracoval se správnou wallet.
```

4. ELECTRUM MUSÍ PODPOROVAT VÍCE SOUČASNĚ OTEVŘENÝCH WALLETŮ

Současný `ElectrumWallet::loadWallet()` dělá přibližně:

```
$loaded = $this->rpc->call('list_wallets');

foreach ($loadedPaths as $path) {
    if ($path !== $walletPath) {
        $this->rpc->call('close_wallet', [
            'wallet_path' => $path
        ]);
    }
}

$this->rpc->call('load_wallet', ...);
```

Toto chování odstranit.

Electrum daemon umí udržovat více walletů současně.

Cílový stav:

```
Electrum daemon
|
├─ wallet_A LOADED
├─ wallet_B LOADED
├─ wallet_C LOADED
├─ wallet_D LOADED
└─ wallet_E LOADED
```

Nikdy při obsluze wallet A automaticky nezavírat wallet B.

Implementuj například:

```
final class ElectrumWalletManager
{
    public function ensureLoaded(string $walletPath): void;

    public function isLoaded(string $walletPath): bool;

    public function close(string $walletPath): void;
}
```

`ensureLoaded()` má:

1. zjistit, zda wallet již existuje mezi `list_wallets`,
2. pokud ano, nic nedělat,
3. pokud ne, zavolat `load_wallet`,
4. nikdy nezavírat ostatní wallety.

Případně lze implementovat efektivnější variantu:

```
zkus wallet-scoped command
↓
wallet not loaded?
↓
load_wallet()
↓
retry přesně jednou
```

Pokud je to kompatibilní s konkrétní verzí Electra.

Wallet se má zavírat pouze explicitně například při:

```
delete store
maintenance
změna konfigurace
ruční unload
shutdown
```

Ne při běžném requestu.

5. WALLET PATH JE BEZPEČNOSTNĚ CITLIVÝ

Protože wallet path bude předáván explicitně RPC commandům, nikdy nesmí být přebírán přímo od klienta.

Zakázáno:

```
$walletPath = $_GET['wallet'];
```

Správný model:

```
API key / store ID
↓
server-side konfigurace / DB
↓
```

```
ověřený wallet ID
    ↓
WalletPathResolver
    ↓
kanonická cesta pod povoleným wallet directory
```

Vytvoř například:

```
final class WalletPathResolver
{
    public function resolve(string $walletName): string;
}
```

Použij:

```
basename()
realpath()
```

a ověř, že výsledná cesta leží uvnitř definovaného:

```
wallet_directory
```

Nikdy nepovol path traversal:

```
../..//...
```

Klient může posílat maximálně logický identifikátor wallet/store, nikoliv filesystem path.

6. ROZDĚL ELECTRUM COMMANDY PODLE TYPU

Je potřeba rozlišovat minimálně:

Daemon commands

Například:

```
list_wallets
load_wallet
close_wallet
version
```



```
version_info  
getinfo
```

Wallet-scoped commands

Například:

```
getbalance  
createnewaddress  
listaddresses  
listunspent  
onchain_history  
payto  
getmpk  
getseed  
getmasterprivate  
add_request
```

Ty musí vždy dostat explicitní wallet context.

Walletless/network commands

Například:

```
getaddressbalance  
getaddresshistory  
getaddressunspent  
broadcast
```

a lokální příkazy typu:

```
validateaddress  
deserialize
```

podle konkrétní verze Electra.

Nepoužívej `requireWalletLoaded()` tam, kde není wallet skutečně potřeba.

7. getAddressBalance NESMÍ VYŽADOVAT WALLET

Současný kód:

```

public function getAddressBalance(string $address): array
{
    $this->requireWalletLoaded();

    $balance = $this->rpc->call(
        'getaddressbalance',
        ['address' => $address]
    );

    ...
}

```

je potřeba změnit.

`getaddressbalance` je v Electru walletless network command.

Proto má vzniknout například:

```

interface BlockchainProviderInterface
{
    public function getAddressObservation(
        string $address
    ): AddressPaymentObservation;
}

```

a:

```

final class ElectrumBlockchainProvider
    implements BlockchainProviderInterface
{
    public function __construct(
        private ElectrumRPC $rpc
    ) {}

    public function getAddressObservation(
        string $address
    ): AddressPaymentObservation
    {
        // network/walletless RPC
    }
}

```

Kontrola blockchainu potom nebude součástí:

ElectrumWallet

ale:

```
BlockchainProvider
```

To připraví systém také na budoucí:

```
EsploraProvider  
BitcoinNodeProvider  
NBXplorerProvider  
jiný vlastní indexer
```

bez změny invoice logiky.

8. BtcInvoiceManager ODPOJIT OD ElectrumWallet

Současný:

```
private ElectrumWallet $wallet;
```

je příliš těsná vazba.

`BtcInvoiceManager` dnes používá wallet hlavně pro:

```
$this->wallet->getNewAddress();  
$this->wallet->getAddressBalance($address);
```

Tyto dvě odpovědnosti oddělit.

Ideálně:

```
final class BtcInvoiceManager  
{  
    public function __construct(  
        private AddressGeneratorInterface $addressGenerator,  
        private BlockchainProviderInterface $blockchain,  
        private string $secretKey,  
        private ?Database $db = null  
    ) {}  
}
```

Ještě lepší může být rozdělení:

```
InvoiceCreator
InvoiceRepository
PaymentStatusService
```

ale není nutné přepisovat vše najednou.

Prioritou je odstranit přímou závislost invoice logiky na `ElectrumWallet`.

9. ADDRESS GENERATOR ABSTRAKCE

Vytvoř:

```
interface AddressGeneratorInterface
{
    public function generate(
        AddressGenerationContext $context
    ): GeneratedAddress;
}
```

DTO:

```
final class GeneratedAddress
{
    public function __construct(
        public readonly string $address,
        public readonly string $source,
        public readonly ?int $index = null,
        public readonly ?string $derivationPath = null
    ) {}
}
```

Například:

```
source = xpub
index = 157
path = 0/157
```

nebo:

```
source = electrum
index = null
```

Implementace:

```
final class XpubAddressGenerator
    implements AddressGeneratorInterface
{
}

final class ElectrumAddressGenerator
    implements AddressGeneratorInterface
{
}
```

a:

```
final class AddressGeneratorFactory
{
    public function forStore(
        Store $store,
        ?string $requestedSource = null
    ): AddressGeneratorInterface;
}
```

10. XPUB MÁ BÝT VÝCHOZÍ ADDRESS GENERATOR

Store má mít nastavení například:

```
address_source = xpub

xpub / descriptor
derivation_scheme
script_type
network
next_address_index

wallet_path
```

Preferuji ukládat dostatek informací, aby derivace nebyla nejednoznačná.

Nepředpokládej, že samotný textový prefix `xpub` vždy jednoznačně určuje typ výsledné adresy.

Preferuj buď:

```
output descriptor
```

nebo explicitně:

```
extended_public_key
account_derivation_path
script_type
network
```

Například pro native SegWit BIP84:

```
account:
m/84'/0'/0'

server derives:
0/index
```

Server musí mít pouze account-level extended public key.

Nikdy se nesnaž derivovat hardened child z xpub.

Příklad:

```
account xpub
↓
0/157
↓
public key
↓
P2WPKH
↓
bc1q...
```

Privátní klíče nejsou pro příjem plateb potřeba.

11. NEIMPLEMENTUJ BIP32 / SECP256K1 / BECH32 RUČNĚ

Pro XPUB derivaci použij důvěryhodnou a udržovanou Bitcoin knihovnu.

Před přidáním nové Composer dependency ověř:

```
aktuální maintenance
Bitcoin mainnet/testnet podpora
BIP32
P2WPKH
```

```
descriptor/xpub podpora  
PHP kompatibilitu
```

Nepřepisuj kryptografii vlastní implementací.

Přidej známé test vectors a porovnej adresy s Electrum/Sparrow/hardware wallet.

Test musí pro konkrétní testovací xpub ověřit například:

```
0/0  
0/1  
0/2  
0/100
```

proti referenční wallet.

12. XPUB INDEX MUSÍ BÝT ATOMICKÝ

Nikdy:

```
$index = $store['next_address_index'];  
$store['next_address_index']++;
```

bez transakce.

Při 100 současných requestech nesmí dva requesty dostat stejný index.

Použij DB transakci:

```
START TRANSACTION;  
  
SELECT next_address_index  
FROM stores  
WHERE id = ?  
FOR UPDATE;
```

potom:

```
index = next_address_index  
next_address_index = index + 1
```

a:

```
UPDATE stores
SET next_address_index = ?
WHERE id = ?;

COMMIT;
```

Výsledkem při 100 paralelních requestech musí být například:

```
request 1   → 1001
request 2   → 1002
request 3   → 1003
...
request 100 → 1100
```

Nikdy:

```
A → 1001
B → 1001
```

DB lock držet pouze po dobu rezervace indexu.

Samotnou kryptografickou derivaci adresy lze dělat mimo DB lock, pokud je index již bezpečně rezervován.

13. STATELESS REŽIM A XPUB

Stateless invoice může zůstat bez databázového záznamu faktury.

XPUB generator ale nemůže být úplně bez stavu, protože musí znát:

```
next_address_index
```

Vytvoř abstrakci:

```
interface AddressIndexStoreInterface
{
    public function reserveNextIndex(
        string $generatorId
    ): int;
}
```

Implementace:


```
DatabaseAddressIndexStore  
FileAddressIndexStore
```

Databázová varianta použije transakční locking.

File varianta může mít například:

```
/var/lib/btcpaylite/address-index/<client>.counter
```

a použít:

```
flock($fp, LOCK_EX);
```

ALE:

lock smí být držen pouze během:

```
read index  
increment index  
write index  
fsync/flush  
unlock
```

tedy velmi krátce.

Nesmí se zamykat celý HTTP request.

14. ODSTRANIT GLOBÁLNÍ LOCK V api_stateless.php

Současné:

```
$lockFile =  
    sys_get_temp_dir() .  
    '/btcpay_electrum_stateless.lock';  
  
flock($fp, LOCK_EX);
```

obepíná prakticky celý request.

To znamená:

```
100 requestů
↓
1 globální fronta
```

Tento globální lock odstranit.

Používat pouze jemně granularizované locky:

```
XPUB:
  per-generator index lock
  pouze několik ms

Electrum address creation:
  per-wallet lock

blockchain status:
  žádný wallet lock

webhooks:
  žádný Electrum lock
```

15. ODSTRANIT GLOBÁLNÍ MySQL GET_LOCK('electrum_rpc')

Současný `api.php` a `webhook_cron.php` používají:

```
SELECT GET_LOCK('electrum_rpc', 10)
```

To serializuje celý systém.

Navíc současný kód výsledek `GET_LOCK()` nekontroluje.

MySQL vrací:

```
1    lock získán
0    timeout
NULL chyba
```

Nikdy nesmí program pokračovat, pokud lock nebyl získán.

Globální:

```
electrum_rpc
```

odstranit.

Pokud je application-level locking potřeba pro mutující wallet operaci, použít:

```
electrum_wallet_<hash wallet id>
```

Například:

```
$lockName =  
    'electrum_wallet_' .  
    hash('sha256', $walletId);
```

a vždy:

```
$result = ...GET_LOCK...;  
  
if ((int)$result !== 1) {  
    throw new ServiceUnavailableException(...);  
}
```

Použít krátký timeout.

Pokud je konkrétní Electrum wallet busy, je lepší vrátit:

```
HTTP 503  
Retry-After: 1
```

než držet PHP-FPM worker desítky sekund.

16. PER-WALLET LOCKING

Application-level exclusive lock používej pouze tam, kde operace mění stav wallet nebo kde chceme dodatečnou ochranu.

Například:

```
createnewaddress  
payto / transaction creation  
signing
```

```
wallet password operations
explicitní wallet maintenance
```

Model:

```
wallet_A → lock_A
wallet_B → lock_B
wallet_C → lock_C
```

Pak:

```
100 requestů na wallet A
→ serializují se mezi sebou

requesty na wallet B
→ pokračují nezávisle

requesty na wallet C
→ pokračují nezávisle
```

Nikdy:

```
A
B
C
D
→ jeden globální Electrum lock
```

Electrum samotné má interní locking pro správu mapy walletů a deterministické vytváření adres. Application-level per-wallet lock je další obranná vrstva, ne náhrada interní synchronizace Electra.

17. ELECTRUM ADDRESS GENERATOR

Implementace může být přibližně:

```
final class ElectrumAddressGenerator
    implements AddressGeneratorInterface
{
    public function __construct(
        private ElectrumWalletManager $walletManager,
        private ElectrumRPC $rpc,
        private WalletLockManager $locks,
        private string $walletPath
    ) {}
```

```

public function generate(
    AddressGenerationContext $context
): GeneratedAddress {
    return $this->locks->withWalletLock(
        $this->walletPath,
        function () {
            $this->walletManager
                ->ensureLoaded($this->walletPath);

            $address = $this->rpc->callWallet(
                $this->walletPath,
                'createnewaddress'
            );

            return new GeneratedAddress(
                address: (string)$address,
                source: 'electrum'
            );
        }
    );
}

```

Lock skončí okamžitě po získání adresy.

Nesmí pod ním probíhat:

```

DB INSERT invoice
webhook creation
checkout URL generation
JSON response
fiat API
další HTTP request

```

18. API VOLBA ADDRESS SOURCE

Výchozí:

```
store.address_source
```

například:

```
xpub
```

Normální request nemusí uvádět nic:

```
{  
  "amount": 0.001  
}
```

Použije:

```
xpub
```

Volitelně je možné povolit například:

```
{  
  "amount": 0.001,  
  "addressSource": "electrum"  
}
```

ALE pouze tam, kde je klient autorizovaný tuto volbu měnit.

Není bezpečné dovolit každému veřejnému klientovi libovolně přepínat backend.

Preferuj:

```
store configuration
```

a API override pouze pro administrátora nebo explicitně povolený API key.

Nedělej automatický fallback:

```
XPUB failure  
→ automaticky Electrum
```

Pokud XPUB selže:

```
ERROR
```

Electrum použij pouze explicitně.

Důvod: systém musí vždy vědět, odkud invoice adresa vznikla.

19. DO INVOICE UKLÁDAT PŮVOD ADRESY

Do DB invoice přidat například:

```
address_source  
address_index  
derivation_path
```

Příklad:

```
invoice_id: inv_123  
btc_address: bclq...  
address_source: xpub  
address_index: 157  
derivation_path: 0/157
```

Electrum:

```
invoice_id: inv_456  
btc_address: bclq...  
address_source: electrum  
address_index: NULL
```

Invoice ↔ adresa mapping musí být trvalý.

Bitcoin adresy nikdy nerecyklovat.

Ani po:

```
expired  
cancelled  
failed
```

Pozdní platba musí stále být přiřaditelná původní invoice.

20. ZPĚTNÁ KOMPATIBILITA STATELESS TOKENŮ

Pokud se bude stateless token rozšiřovat o:

```
address_source
address_index
version
```

zaved' verzi payloadu.

Například:

```
{
  "ver": 2,
  "a": "bc1...",
  "v": "0.00100000",
  "s": "xpub",
  "i": 157,
  "d": "...",
  "p": {},
  "t": 123,
  "e": 456
}
```

`decodeStatelessToken()` musí stále umět načíst existující staré tokeny bez `ver`.

Nesmí dojít k okamžitému zneplatnění existujících faktur.

21. PAYMENT STATUS NESMÍ BÝT SPOUŠTĚN KAŽDÝM BROWSER REQUESTEM

Toto je jedna z nejdůležitějších změn.

Představ si:

```
1000 otevřených payment pages
poll každých 5 sekund
```

To znamená:

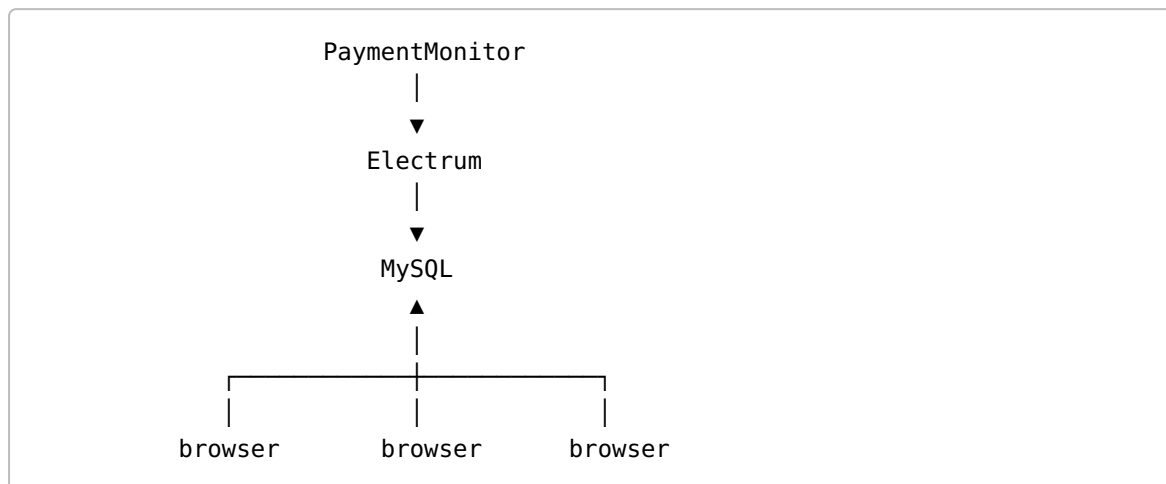
```
200 HTTP status requestů / s
```

Pokud každý udělá Electrum RPC:

```
200 RPC/s
```


systém bude zbytečně přetěžován.

Cílový DB režim:



Browser:

```
GET /invoice/.../status
```

má dělat pouze:

```
SELECT ...  
FROM invoices  
WHERE id = ?
```

Nikdy přímo:

```
Electrum RPC
```

22. PAYMENT MONITOR WORKER

Současný `webhook_cron.php` přestav na jasnější background worker architecture.

Například:

```
payment_worker.php  
webhook_worker.php
```

Payment worker:

1. najde invoices, které potřebují kontrolu
2. vezme batch
3. zkontroluje blockchain
4. aktualizuje stav v DB
5. vytvoří webhook event/outbox záznam
6. pokračuje

Invoice může mít:

```
last_checked_at
next_check_at
check_attempts
payment_confirmed_amount
payment_unconfirmed_amount
payment_last_seen_at
```

Polling interval může být adaptivní.

Příklad:

```
nová invoice:
    každých několik sekund

starší aktivní invoice:
    každých 10–30 sekund

settled:
    přestat aktivně pollovat

expired:
    výrazně méně často
    ale zachovat late-payment detection
```

Přidej malý jitter, aby stovky invoice nebyly kontrolovány přesně ve stejný okamžik.

Například:

```
9.1 s
10.7 s
8.9 s
11.2 s
```

23. WORKER MUSÍ BÝT SCHOPNÝ ŠKÁLOVAT

Pokud běží více payment workerů, nesmí kontrolovat stejnou invoice současně.

Pro MySQL 8 lze zvážit:

```
SELECT ...  
FOR UPDATE SKIP LOCKED
```

Pokud verze DB `SKIP LOCKED` nepodporuje, implementuj lease mechanismus:

```
processing_by  
processing_until
```

Worker atomicky claimne batch.

Neimplementuj to naslepo; zjisti MySQL/MariaDB verzi projektu.

24. STATELESS PAYMENT STATUS POTŘEBUJE CACHE A SINGLE-FLIGHT

Stateless faktura nemá hlavní DB record.

Proto použij krátkodobou cache.

Například:

```
key:  
hash(invoice token)  
nebo address  
  
value:  
confirmed_received  
unconfirmed_received  
status  
checked_at  
expires_at
```

TTL například několik sekund.

Důležitý concurrency mechanismus:

100 requestů stejného statusu

nesmí znamenat:

100 Electrum calls

Použij model:

```
request 1
→ cache miss
→ získá krátký per-address status lock
→ Electrum
→ zapíše cache

request 2–100
→ čekají krátce / znovu načtou cache
→ žádný další Electrum request
```

Tedy tzv. request coalescing / single-flight.

Rozhraní:

```
interface PaymentStatusCacheInterface
{
    public function get(string $key): ?PaymentObservation;

    public function put(
        string $key,
        PaymentObservation $value,
        int $ttlSeconds
    ): void;
}
```

Implementace může být zpočátku jednoduchá.

Není nutné zavádět Redis, pokud systém může bezpečně použít MySQL/file cache.

Architektura ale má umožnit Redis přidat později.

25. BLOCKCHAIN PROVIDER NESMÍ V KONEČNÉM STAVU POUŽÍVAT POUZE CURRENT BALANCE

Současný systém rozhoduje přibližně:

confirmed + unconfirmed

z aktuálního balance adresy.

To není ideální dlouhodobý účetní model.

Pokud budou prostředky z dané adresy později utráceny, current balance může klesnout, přestože invoice byla historicky zaplacená.

Proto finální `BlockchainProvider` má vracet skutečnou informaci o platbách přijatých na invoice adresu.

Například:

```
final class AddressPaymentObservation
{
    public function __construct(
        public readonly int $confirmedReceivedSats,
        public readonly int $unconfirmedReceivedSats,
        public readonly array $transactions,
        public readonly int $observedAt
    ) {}
}
```

Payment status se má odvozovat z:

outputs / transaction history / received amount

ne pouze z momentálního spendable balance.

Pokud je toto příliš velká změna pro první refaktor, zachovej současnou balance logiku dočasně, ale vytvoř nové rozhraní tak, aby bylo možné implementaci následně opravit bez změny invoice manageru.

26. BTC ČÁSTKY NEPOČÍTAT PŘES FLOAT

Současný systém používá na několika místech:

float

pro BTC amount.

Pro finanční backend preferuj interně:

```
integer satoshis
```

tedy:

```
int $amountSats;
```

Na hranici API:

```
"0.00123456"
```

bezpečně převést na:

```
123456 sats
```

bez binární floating-point chyby.

Výstup lze zpět formátovat jako BTC decimal string.

Pokud je tato změna příliš rozsáhlá, udělej ji samostatnou migrační etapou, ale nově vytvořené DTO a služby navrhní tak, aby pracovaly v satoshi.

27. WEBHOOKY ODDĚLIT OD PAYMENT MONITORU

Payment worker nesmí čekat na externí webhook server.

Aktuální model posílá webhook relativně přímo po změně statusu.

Zaved' ideálně outbox:

```
payment worker
  |
  ▼
UPDATE invoice
+
INSERT webhook_outbox
  |
  ▼
COMMIT
```

Samostatný:

```
webhook_worker
```

odesílá:

```
webhook_outbox
```

a řeší:

```
retry  
timeout  
attempt count  
next_attempt_at  
last_error  
delivered_at
```

Použij exponential backoff.

Například:

```
1 min  
5 min  
15 min  
1 h  
...
```

Konkrétní hodnoty nastav rozumně konfigurovatelně.

Pomalý nebo mrtvý webhook endpoint nesmí blokovat:

```
create invoice  
payment status  
Electrum  
ostatní klienty
```

28. IDEMPOTENCY CREATE INVOICE

Je nutné řešit tento scénář:

```
client  
→ POST create invoice  
→ server invoice vytvoří  
→ response se cestou ztratí
```

```
→ client timeout  
→ zopakuje POST
```

Bez idempotency:

```
invoice A + adresa A  
invoice B + adresa B
```

pro jednu objednávku.

Implementuj podporu:

```
Idempotency-Key: ...
```

nebo bezpečně použij existující:

```
order_id
```

v kombinaci se store.

Například DB unique key:

```
(store_id, idempotency_key)
```

Stejný request se stejným key musí vrátit původní invoice.

Nikdy nesmí rezervovat další XPUB index.

Pokud stateless režim chce skutečnou idempotency napříč requesty, musí pro tuto část uchovávat minimální stav. Úplně stateless systém nemůže garantovat idempotentní create napříč nezávislými requesty bez nějaké persistence.

29. BACKPRESSURE – SYSTÉM SE NESMÍ PŘI PŘETÍŽENÍ ZHROUTIT

Nechci systém navrhnut tak, že při 100 nebo 1000 současných requestech všechny čekají na Electrum 30 sekund a obsadí PHP-FPM.

Použij princip:


```
FAST PATH
DB
cache
XPUB
```

a:

```
SLOW PATH
Electrum
```

musí být omezený.

Pokud Electrum wallet operace nemůže získat per-wallet lock během krátkého času:

```
503 Service Unavailable
Retry-After: 1
```

je lepší než dlouhé blokování workeru.

RPC timeouty rozděl podle typu operace.

Současný blanket:

```
CURLOPT_TIMEOUT => 30
```

není ideální pro všechny commandy.

Například:

```
connect timeout:
krátký

běžný read:
krátký

wallet maintenance / speciální write:
delší
```

Konkrétní čísla nastav jako config, ne natvrdo v každé třídě.

30. CIRCUIT BREAKER

Připrav systém na situaci:

Electrum daemon down

Nesmí nastat:

1000 requests
→ 1000× timeout do Electra
→ PHP-FPM kolaps

Implementuj jednoduchý circuit breaker nebo alespoň centralizované health state.

Například:

5 po sobě jdoucích RPC failures
→ Electrum marked unhealthy na několik sekund

Během této doby:

Electrum-specific operation
→ okamžitě 503

XPUB create invoice ale může nadále fungovat.

DB status endpoint může vracet:

last known status

s údajem:

last_checked_at

Nemusí kvůli nedostupnému Electru přestat fungovat celý web.

31. XPUB MODE MUSÍ PŘEŽÍT VÝPADEK ELECTRA

To je jedna z hlavních výhod nové architektury.

Pokud:

Electrum DOWN

musí být stále možné:

```
login
admin
DB reads
create invoice přes XPUB
zobrazit existující invoice
vrátit poslední známý payment status
```

Nefunkční budou pouze operace skutečně závislé na Electru.

To ověř integračním testem.

32. ELECTRUM MODE MÁ ZŮSTAT PLNĚ FUNKČNÍ

Neodstraňuj:

```
ElectrumRPC
ElectrumWallet
wallet admin
wallet balance
history
send transaction
manual address generation
```

pokud není nezbytné jejich interní API změnit.

Cílem je:

```
XPUB = výchozí moderní fast path
Electrum = alternativní / kompatibilní wallet backend
```

Ne odstranit původní funkcionalitu.

33. READ A WRITE OPERACE ELECTRA NEMAJÍ STEJNÉ LOCKY

Navrhni klasifikaci:

```
READ / NETWORK
→ žádný exclusive wallet lock
```

WALLET READ

- explicitní wallet context
- typicky bez application exclusive lock

WALLET WRITE / MUTATION

- per-wallet exclusive lock

Například:

getaddressbalance

- žádná wallet
- žádný wallet lock

getbalance(wallet A)

- wallet A
- bez write locku

createnewaddress(wallet A)

- wallet A
- per-wallet write lock

payto(wallet A)

- wallet A
- per-wallet write lock

Interní locking Electra respektuj.

Neobaluj všechno jedním zámekem „pro jistotu“, protože tím zničíš paralelismus.

34. PAYMENT MONITOR NEMÁ VOLAT loadWallet()

Současný `webhook_cron.php` dělá přibližně:

GET_LOCK

loadWallet(invoice wallet)

check payment

RELEASE_LOCK

Pro address-level blockchain query to není potřeba.

Nově:

```
invoice
  ↓
btc_address
  ↓
BlockchainProvider
  ↓
network query
```

Wallet path není pro běžné ověření platby potřeba.

To znamená, že invoice klient A a klient B se při payment monitoringu nemusí navzájem blokovat pouze proto, že mají jiné Electrum wallet soubory.

35. BtcStatelessService::checkStatus() NEMÁ loadovat WALLET

Současný kód dekoduje token, najde:

```
p.wallet
```

a před kontrolou statusu zavolá:

```
$this->wallet->loadWallet($walletPath);
```

To odstranit pro payment status.

Status potřebuje:

```
address
amount
expiration
```

a BlockchainProvider.

Ne wallet.

Wallet metadata lze v tokenu zachovat kvůli kompatibilitě/diagnostice, ale payment check ji nemusí používat.

36. url_pay.php A FRONTEND POLLING

Platební stránka nemá každých několik sekund spouštět těžkou Electrum operaci.

DB invoice:

```
frontend
→ vlastní status endpoint
→ DB
```

Stateless invoice:

```
frontend
→ vlastní status endpoint
→ krátká cache
→ maximálně občas Electrum provider
```

Nikdy:

```
frontend
→ externí blockchain API přímo
```

Externí provider musí být schovaný za serverem.

37. SYSTÉM MUSÍ BÝT PŘIPRAVEN NA VÍCE ELECTRUM DAEMONŮ, ALE NENÍ TO PRVNÍ KROK

První cílová konfigurace:

```
1 Electrum daemon
|
├─ wallet 1
├─ wallet 2
├─ wallet 3
├─ ...
└─ wallet N
```

Pokud benchmark později ukáže, že jeden daemon je bottleneck, architektura má umožnit:

```
Electrum daemon #1
→ stores 1–50
```

```
Electrum daemon #2  
→ stores 51–100
```

```
Electrum daemon #3  
→ stores 101–150
```

tedy sharding.

Proto store/wallet konfigurace nemá předpokládat jeden globální:

```
rpc_host  
rpc_port
```

Navrhni možnost budoucího:

```
electrum_backend_id
```

nebo backend config per store/group.

Není nutné implementovat plný sharding okamžitě, ale nové vrstvy ho nesmějí znemožnit.

38. NEDĚLAT ROVNOU 1 ELECTRUM DAEMON NA KAŽDÉHO KLIENTA

To by sice přineslo izolaci:

```
client A → daemon A  
client B → daemon B
```

ale provozně znamená:

```
N× port  
N× process  
N× config  
N× systemd service  
N× log  
N× monitoring  
N× connection management
```

Proto nejdříve:

```
1 daemon
+
multi-wallet
+
explicitní wallet routing
+
per-wallet lock
```

A teprve podle benchmarku shardovat.

39. BEZPEČNOST ELECTRUM WALLETŮ

Pro receiving/payment infrastrukturu preferuj:

```
watch-only wallet
```

ne:

```
hot wallet se seedem/xprv
```

Ideální:

```
web server
|
├── XPUB / descriptor
└── watch-only Electrum
```

Privátní klíče:

```
hardware wallet
offline signer
jiný oddělený secure systém
```

Pokud server drží seed/xprv všech klientů, kompromitace serveru může znamenat kompromitaci všech prostředků.

Proto při refaktoru zkontroluj také současné admin funkce, které umožňují:

```
getSeed()
getMasterPrivateKey()
export keys
```


Pro produkční režim musí být tato funkcionality buď odstraněna, nebo explicitně deaktivovatelná bezpečnostním configem a nesmí být běžně dostupná přes webové rozhraní.

40. DŮVĚRYHODNOST `getaddressbalance`

Electrum u walletless:

```
getaddressbalance  
getaddresshistory
```

upozorňuje, že výsledky nejsou SPV-ověřené.

Proto:

Pokud je Electrum daemon napojený na náhodný veřejný Electrum server:

```
server tvrdí, že invoice zaplacená
```

nesmí být automaticky považováno za nejvyšší možnou úroveň důvěry pro nevratné vyplacení fiat prostředků.

Pro produkční směnárnou preferuj:

```
vlastní / trusted Electrum server
```

ideálně infrastrukturu napojenou na vlastní Bitcoin node.

Architektura `BlockchainProvider` musí umožnit pozdější přechod na:

```
own Electrum server  
Bitcoin Core + indexer  
NBXplorer  
jiný trusted backend
```

bez změny invoice systému.

41. WEBHOOK DELIVERY A PAYMENT STATUS MUSÍ BÝT IDEMPOTENTNÍ

Při změně statusu například:

```
New → Processing
```

nesmí dva workery vytvořit nekontrolovaně dva stejné webhook eventy.

Použij unique event key například:

```
invoice_id + event_type + transition/version
```

nebo jinou transakčně bezpečnou metodu.

Webhook worker může konkrétní event retryovat, ale nesmí náhodně generovat nové logické eventy při každém pokusu.

42. OBSERVABILITY

Přidej strukturované logování alespoň pro:

```
request_id
store_id
invoice_id
wallet_id
address_source
electrum_method
RPC duration
lock wait duration
cache hit/miss
payment check duration
webhook attempt
error category
```

Nikdy do logů nekládat:

```
seed
xprv
RPC password
API key
full authorization header
```

Potřebuji být schopný zjistit například:

```
proč invoice creation trvalo 4 sekundy?
```

a vidět:

```
xpub derive: 2 ms  
DB: 8 ms  
nebo  
Electrum lock wait: 3200 ms
```

43. HEALTH CHECK

Přidej interní health check například:

```
DB  
Electrum daemon  
Electrum network connection  
payment worker heartbeat  
webhook worker heartbeat
```

Oddělit:

```
liveness
```

od:

```
readiness
```

XPUB API nemusí být úplně unavailable jen proto, že payment monitor právě nemá Electrum.

44. API RATE LIMITING

Vedle správné concurrency architektury přidej možnost rate limitu alespoň podle:

```
API key  
store  
IP pro veřejné endpointy
```

Ne kvůli běžnému provozu, ale kvůli tomu, aby jeden klient nemohl obsadit všechny PHP workers.

Limit musí být konfigurovatelný.

Neřeš výkon pouze locky.

45. PHP-FPM / HTTP BACKPRESSURE

Aplikace musí počítat s omezeným počtem PHP-FPM workerů.

Nedrž worker desítky sekund při:

```
lock čekání  
Electrum timeout  
webhook HTTP timeout  
externí fiat API
```

HTTP request má být rychlý.

Pomalé práce přesouvat do workerů.

V dokumentaci projektu doplň doporučení pro:

```
PHP-FPM max_children  
Nginx/Apache timeouts  
request limits  
worker deployment
```

Konkrétní hodnoty neurčuj bez benchmarku konkrétního serveru.

46. FIAT PRICE A EXTERNÍ API TAKÉ CACHEOVAT

Současný dashboard/payment page používá externí fiat price API.

Ani fiat price nesmí být volán:

```
1 browser = 1 externí API call
```

Přidej krátkou cache například na desítky sekund podle potřeby.

Výpadek fiat price providera nesmí shodit BTC payment processing.

47. DATABÁZOVÁ MIGRACE

Navrhni konkrétní SQL migration.

Minimálně zvaž přidání do `stores`:

```
address_source
extended_public_key / descriptor
derivation_path
script_type
network
next_address_index
```

podle finálního návrhu.

Do `invoices`:

```
address_source
address_index
derivation_path

confirmed_received_sats
unconfirmed_received_sats

last_checked_at
next_check_at
check_attempts

idempotency_key
```

Do webhook infrastruktury případně:

```
webhook_outbox
```

Při migraci:

```
stávající store
→ address_source = electrum
```

aby po deploymentu nedošlo ke změně chování existujících instalací bez explicitního nastavení.

Až poté může administrátor nastavit:

```
xpub
```

jako nový default pro konkrétní store nebo globálně pro nové stores.

To je důležité pro bezpečnou migraci.

48. XPUB KONFIGURACI PŘED AKTIVACÍ OVĚŘIT

Při uložení XPUB/deskriptoru:

1. validovat formát,
2. validovat network,
3. validovat script type,
4. derivovat několik testovacích adres,
5. umožnit administrátorovi porovnat například první adresu s wallet,
6. teprve potom backend aktivovat.

Špatný xpub/derivation path znamená, že zákazníci posílají BTC na adresy, které provozovatel neumí utratit.

Proto konfigurace nesmí být přijata bez validace.

49. DOPORUČENÝ FINÁLNÍ CLASS DESIGN

Nemusí být názvy přesně stejné, ale odpovědnosti mají být podobné:

```
classes/

ElectrumRPC.php
ElectrumRpcDialect.php
ElectrumWalletManager.php
ElectrumWallet.php
WalletPathResolver.php
WalletLockManager.php

AddressGeneratorInterface.php
GeneratedAddress.php
AddressGenerationContext.php
XpubAddressGenerator.php
ElectrumAddressGenerator.php
AddressGeneratorFactory.php

AddressIndexStoreInterface.php
DatabaseAddressIndexStore.php
FileAddressIndexStore.php

BlockchainProviderInterface.php
ElectrumBlockchainProvider.php
AddressPaymentObservation.php

BtcInvoiceManager.php
InvoiceRepository.php           [pokud dává smysl]
PaymentStatusService.php        [pokud dává smysl]
```

```
PaymentStatusCacheInterface.php
...

IdempotencyService.php
WebhookOutboxRepository.php
```

Nevytvářej zbytečné enterprise abstrakce jen kvůli abstrakcím.

Každá nová vrstva musí řešit konkrétní problém.

50. CÍLOVÝ REQUEST FLOW – XPUB

```
POST create invoice
  |
  ▼
authenticate
  |
  ▼
check idempotency
  |
  ▼
load store config
  |
  ▼
address_source = xpub
  |
  ▼
atomic reserve address index
  |
  ▼
derive BTC address locally
  |
  ▼
INSERT invoice
  |
  ▼
return checkout data
```

Electrum RPC calls:

```
0
```

51. CÍLOVÝ REQUEST FLOW – ELECTRUM GENERATOR

```
POST create invoice
|
▼
authenticate
|
▼
check idempotency
|
▼
store.address_source = electrum
|
▼
resolve trusted wallet path
|
▼
acquire per-wallet lock
|
▼
ensure wallet loaded
|
▼
createnewaddress(wallet)
|
▼
release lock
|
▼
INSERT invoice
|
▼
return response
```

Lock nesmí pokračovat přes DB/webhook/response processing.

52. CÍLOVÝ STATUS FLOW – DATABASE INVOICE

```
browser
|
▼
GET invoice status
|
▼
```



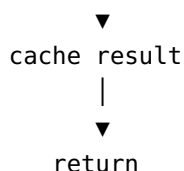
```
MySQL
|
▼
return current cached/persisted status
```

Samostatně:

```
PaymentMonitor
|
▼
find due invoices
|
▼
BlockchainProvider
|
▼
update DB
|
▼
enqueue webhook
```

53. CÍLOVÝ STATUS FLOW – STATELESS

```
browser
|
▼
decode + verify HMAC token
|
▼
PaymentStatusCache
|
├── HIT
│   │
│   ▼
│   return
|
└── MISS
    |
    ▼
    short per-address single-flight lock
    |
    ▼
    check cache again
    |
    ▼
    BlockchainProvider
    |
```



54. LOAD TESTY, KTERÉ MUSÍ BÝT SOUČÁSTÍ ÚPRAVY

Přidej nebo připrav testovací skripty.

Test A – XPUB concurrency

100 současných create invoice
stejný store
XPUB mode

Očekávání:

100 HTTP responses
100 unikátních invoice IDs
100 unikátních BTC adres
100 unikátních derivation indexes
žádný Electrum RPC
žádný duplicate index

Test B – idempotency

100× současně stejný Idempotency-Key

Očekávání:

1 invoice
1 address
1 reserved XPUB index
všech 100 requestů dostane logicky stejný invoice výsledek

Test C – multi-wallet Electrum

Načíst například:

```
wallet A  
wallet B  
wallet C
```

Potom paralelně:

```
create address A  
create address B  
create address C
```

Očekávání:

```
žádné close wallet jiné wallet  
všechny tři zůstávají loaded  
žádný cross-wallet context
```

Test D – wallet isolation

Drž:

```
lock wallet A
```

a současně request:

```
wallet B
```

Wallet B nesmí čekat na lock A.

Test E – status burst

```
100 nebo 1000 současných status requestů  
stejná invoice
```

DB invoice:

0 Electrum calls z HTTP vrstvy

Stateless:

maximálně jeden blockchain query na cache TTL / single-flight cyklus

Test F – Electrum outage

Vypnout Electrum.

Očekávání:

XPUB create invoice → funguje
DB invoice read → funguje
admin/auth/DB → funguje
Electrum address generation → rychlé 503
payment monitoring → zaznamená failure
nezhodí aplikaci

Test G – slow webhook

Webhook server odpovídá například po dlouhé době / neodpoví.

Očekávání:

invoice create není ovlivněno
payment monitor není dlouhodobě blokován
webhook worker retryuje samostatně

55. BENCHMARK

Po implementaci změř:

1
10

```
50
100
200
```

současných requestů minimálně pro:

```
XPUB create invoice
Electrum create invoice
DB status read
stateless status
```

Reportuj:

```
requests/sec
p50
p95
p99
errors
timeouts
Electrum RPC count
DB query count
lock wait time
```

Neodhaduj výkon teoreticky, pokud ho lze změřit.

56. MIGRAČNÍ POŘADÍ

Úpravu nedělej jedním obrovským přepisem.

Postupuj přibližně:

Fáze 1

Přidej testy současného chování.

Fáze 2

Refaktoruj `ElectrumRPC`:

```
odstranit activeWallet
odstranit /?wallet=
explicitní wallet-scoped RPC
```

```
network RPC
daemon RPC
```

Fáze 3

Refaktoruj `ElectrumWallet`:

```
nezavírat ostatní wallety
multi-wallet support
explicitní wallet context
```

Fáze 4

Odděl `BlockchainProvider` od `ElectrumWallet`.

Přesuň:

```
getAddressBalance / payment observation
```

do provideru.

Odstraň z payment status flow:

```
loadWallet()
```

Fáze 5

Odstraň globální Electrum locky.

Zaveď per-wallet locking pouze pro mutující wallet operace.

Fáze 6

Přidej `AddressGeneratorInterface`.

Nejdříve:

```
ElectrumAddressGenerator
```

aby zachoval současné chování.

Fáze 7

Přidej:

```
XpubAddressGenerator  
AddressIndexStore
```

a DB migrace.

Fáze 8

Zaved':

```
payment worker  
status DB/cache
```

aby browser polling nezatěžoval Electrum.

Fáze 9

Odděl webhook worker/outbox.

Fáze 10

Přidej:

```
idempotency  
backpressure  
circuit breaker  
observability
```

Fáze 11

Load test.

Fáze 12

Teprve podle benchmarku rozhodnout, zda:

```
1 Electrum daemon stačí
```

nebo je potřeba:

sharding na více daemonů

57. ZACHOVEJ EXISTUJÍCÍ API KOMPATIBILITU

Projekt již implementuje Greenfield-like endpointy.

Pokud to není nezbytné, neměň jejich veřejný kontrakt.

Nové parametry musí být:

optional

a staré klienty nesmí rozbít.

Případné nové možnosti:

addressSource
Idempotency-Key

implementuj zpětně kompatibilně.

Stejně tak zachovej staré stateless tokeny.

58. NEPŘEPISUJ FUNKČNÍ ČÁSTI BEZ DŮVODU

Nechci „rewrite“.

Chci:

evolutionary refactoring

Používej stávající:

Database
AuthManager
UrlManager
API routing
invoice API
admin


```
stores
webhooks
```

pokud jejich změna není nezbytná.

Hlavní zásahy mají být kolem:

```
ElectrumRPC
ElectrumWallet
address generation
payment checking
locking
worker architecture
```

59. DŮLEŽITÁ BEZPEČNOSTNÍ PRAVIDLA

Při celé implementaci dodrž:

1. nikdy nelogovat seed/xprv/password/API token,
2. wallet path nikdy nepřebírat přímo od klienta,
3. XPUB konfiguraci před použitím validovat,
4. žádné address reuse,
5. žádné hardened derivation z XPUB,
6. žádné vlastní implementace kryptografie,
7. BTC částky preferovat jako integer satoshi,
8. žádný automatický fallback XPUB → Electrum,
9. externí blockchain data považovat za trust boundary,
10. pro produkční payout preferovat trusted/self-hosted blockchain backend,
11. RPC Electrum vystavit pouze lokálně / do důvěryhodné interní sítě,
12. každá write operace musí být idempotentní nebo bezpečně serializovaná.

60. CO OČEKÁVÁM OD AI

Nechci pouze návrh.

Po prostudování projektu:

1. vysvětlí stručně současný dependency graph,
2. identifikuj všechny soubory, které budou změněny,
3. navrhni DB migration,
4. implementuj nové classes/interfaces,
5. uprav existující classes,
6. uprav `api.php`,
7. uprav `api_stateless.php`,

8. uprav `BtcStatelessService`,
9. uprav `BtcInvoiceManager`,
10. uprav `webhook_cron.php` nebo ho nahraď novými workery,
11. uprav admin části, které používají Electrum wallet context,
12. zachovej kompatibilitu,
13. přidej testy,
14. přidej load-test scénáře,
15. aktualizuj README,
16. proved' code review výsledku se zaměřením na concurrency a security.

Před každou změnou zjisti, jak danou funkcionalitu používají ostatní soubory projektu.

Nevytvářej nové API pouze podle názvu metody bez kontroly call sites.

61. AKCEPTAČNÍ KRITÉRIA FINÁLNÍHO SYSTÉMU

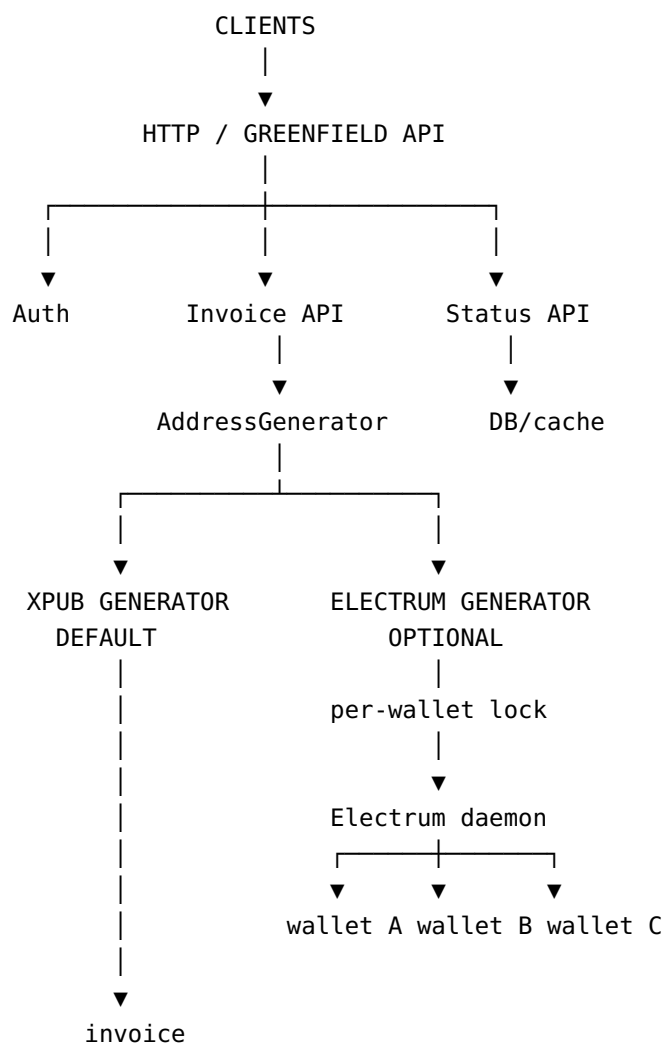
Refaktor je úspěšný pouze pokud platí všechno následující:

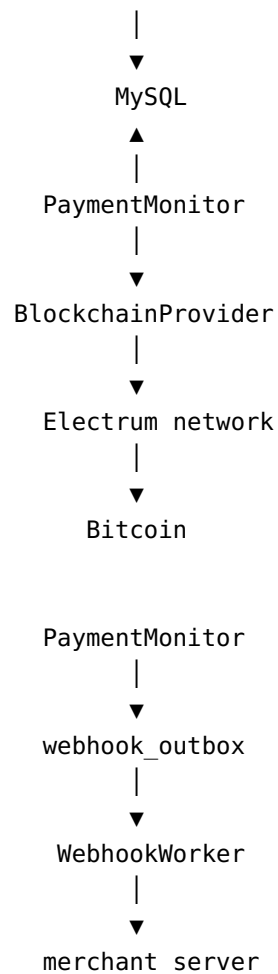
- ✓ více Electrum walletů může zůstat současně loaded
- ✓ request pro wallet A nezavírá wallet B
- ✓ neexistuje globální activeWallet v PHP RPC klientovi
- ✓ wallet-scoped RPC explicitně identifikuje wallet
- ✓ payment address check nepotřebuje loadWallet()
- ✓ neexistuje globální `GET_LOCK('electrum_rpc')`
- ✓ neexistuje globální `btcpay_electrum_stateless.lock` kolem celého requestu
- ✓ write locking je maximálně per-wallet
- ✓ X PUB je podporovaný AddressGenerator
- ✓ Electrum je podporovaný AddressGenerator
- ✓ X PUB index reservation je atomická
- ✓ dvě invoice nikdy nedostanou stejný X PUB index
- ✓ invoice si pamatuje address source + index
- ✓ adresy se nikdy nerecyklují
- ✓ DB status HTTP request nevolá Electrum

- ✓ stateless status má cache/single-flight ochranu
- ✓ webhook delivery neblokuje Electrum ani HTTP
- ✓ create invoice podporuje idempotency
- ✓ výpadek Electra neshodí XPUB část aplikace
- ✓ systém při overloadu používá backpressure místo nekonečného čekání
- ✓ existují concurrency/load testy

62. CÍLOVÝ VÝSLEDEK

Výsledný BTC Pay Lite má vypadat koncepčně takto:





Hlavní filozofie:

HTTP requests jsou rychlé.

XPUB generování je lokální.

Electrum není globální mutex.

Více walletů může být otevřených současně.

Wallet operace jsou explicitně směrované.

Payment monitoring je centralizovaný.

Browser polling čte DB/cache.

Pomalé externí operace běží mimo request path.

Jeden klient nesmí zablokovat ostatní.

Výpadek jedné komponenty nesmí shodit celý systém.

Privátní klíče pokud možno nejsou na payment serveru.

63. DŮLEŽITÉ: NEZAČÍNEJ XPUB IMPLEMENTACÍ DŘÍVE, NEŽ OPRAVÍŠ RPC ARCHITEKTURU

Pořadí je důležité.

Nejdříve oprav:

```
ElectrumRPC
multi-wallet
wallet routing
walletless calls
locking
```

Potom přidej:

```
AddressGenerator
XPUB
```

Důvod je ten, že Electrum zůstává součástí systému a jeho současné globální stavové chování by jinak zůstalo zdrojem race conditions.

64. PŘED KÓDOVÁNÍM PROVEĎ ELECTRUM COMPATIBILITY PROBE

Protože Electrum API se mezi verzemi mění, před finálním návrhem RPC vrstvy:

1. zjisti verzi Electrum,
2. ověř `commands`,
3. ověř `list_wallets`,
4. ověř `load_wallet`,
5. načti dvě testovací watch-only wallets současně,
6. ověř, že obě zůstávají v `list_wallets`,
7. proveď wallet-scoped command zvlášť pro každou,
8. ověř walletless `getaddressbalance`,
9. ověř command pro vytvoření nové adresy,
10. teprve potom zafixuj `ElectrumRpcDialect`.

Pokud Work prostředí nemá přístup k běžícímu daemonu, implementuj kompatibilní adapter a připrav přesný diagnostický CLI/PHP skript, který tyto testy provedu na serveru.

65. NEJDŮLEŽITĚJŠÍ PRINCIP CELÉHO REFAKTORU

Neřeš vysokou zátěž tím, že:

„uděláme Electrum dost rychlé,
aby obsloužilo každý HTTP request“

Řeš ji tím, že:

většina HTTP requestů
Electrum vůbec nepotřebuje.

Typický vysoký provoz po refaktoru může vypadat:

1000 HTTP requestů

900 → DB/cache
80 → lokální XPUB derivace
15 → řízené blockchain checks
5 → skutečné wallet RPC operace

To je architektura, ke které celý systém směřuj.

66. FINÁLNÍ VÝSTUP OD AI

Po dokončení mi vrať:

A. Architektonický souhrn

Co bylo změněno a proč.

B. Seznam změněných/nových souborů

Například:

MODIFIED ...
NEW ...
REMOVED ...

C. DB migration

Kompletní SQL.

D. Významné compatibility změny

Zvlášť:

```
Electrum version handling  
API compatibility  
stateless token compatibility
```

E. Security review

Nalezené problémy a jejich řešení.

F. Concurrency review

Vysvětlí:

```
co se stane při 100 concurrent create invoice  
co se stane při 1000 status requests  
co se stane při busy wallet A  
co se stane při Electrum outage
```

G. Test report

Jednotkové, integrační a load testy.

H. Co zůstává jako další fáze

Například:

```
own Electrum server  
Bitcoin Core/indexer  
Redis  
Electrum sharding  
lepší transaction-level accounting
```

Nenechávej neimplementované kritické race conditions pouze jako TODO.

Pokud některou část nelze bezpečně implementovat bez znalosti konkrétní verze nebo prostředí, implementuj bezpečný adapter/config možnost, zdokumentuj ji a připrav test, kterým se správná varianta na produkčním serveru ověří.